

FIAP
GRADUAÇÃO

COMO RESOLVER

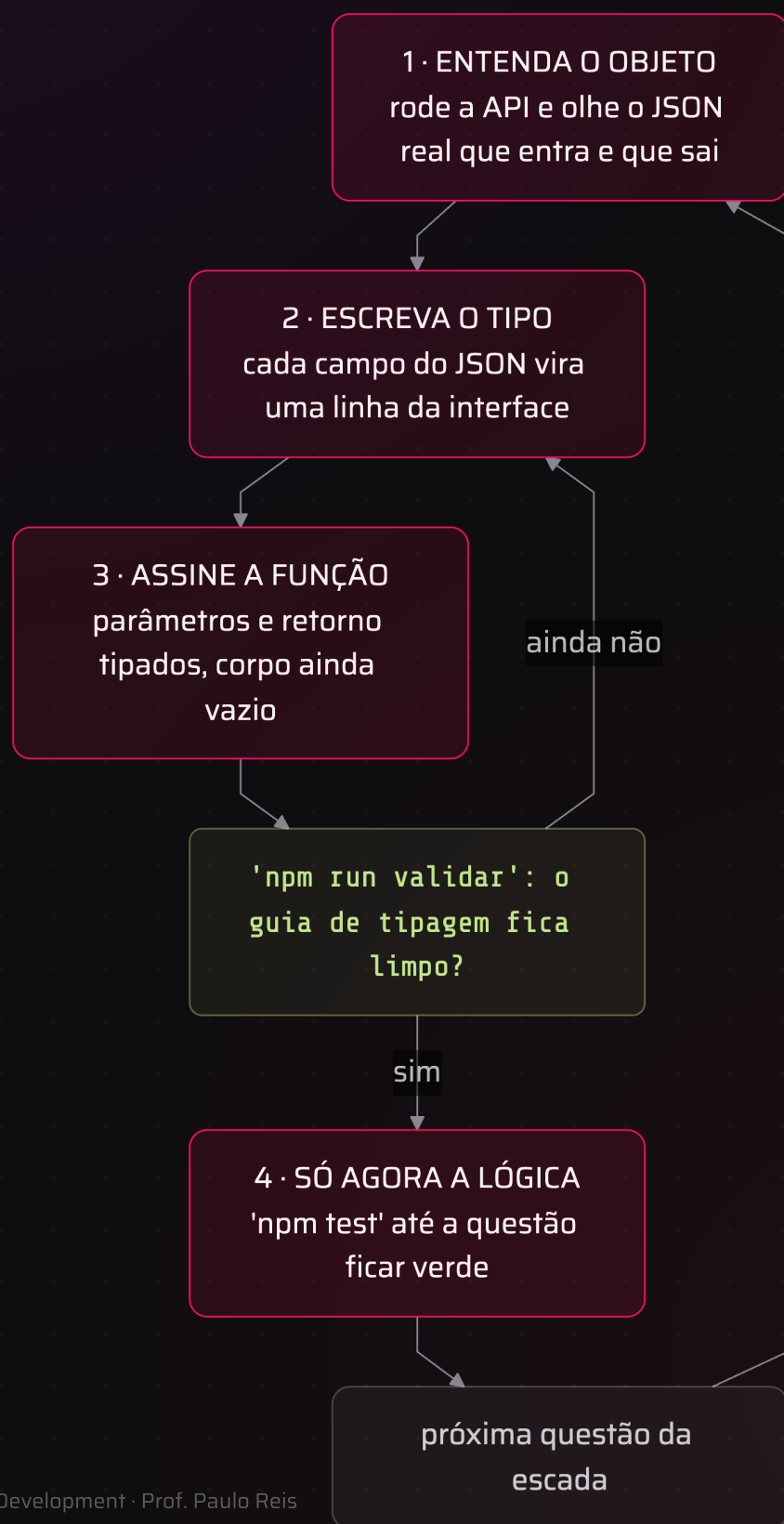
// primeiro a estrutura, depois os dados

o método que a gente sugere para cada questão do Checkpoint

Prof. Paulo Reis

O ciclo inteiro, numa figura

A tentação é abrir o arquivo e sair escrevendo lógica. Segure. Quem escreve o tipo primeiro erra menos e erra mais cedo, quando o erro ainda é barato: é o compilador que aponta, não o teste quebrando meia hora depois.



Passo 1 · entenda o objeto

Antes de tocar no teclado, responda no papel: **que dado entra? que dado sai?**

Você tem três fontes, nesta ordem:

- o **ENUNCIADO**, seção "entra e sai" de cada questão: exemplos reais, letra por letra;
- os **testes** da questão, em `tests/`: cada `expect` mostra uma chamada de verdade;
- o **Swagger** com a API rodando (`npm run dev`): o JSON vivo, do jeito que chega.

Aí descreva o objeto campo a campo. Um exemplo de fora do Checkpoint, um empréstimo de bicicleta:

```
{ "id": "b7", "aro": 29, "elettrica": false, "situacao": "LIVRE" }
```

campo	o que é	tipo
<code>id</code>	identificador, texto	<code>string</code>
<code>aro</code>	número da roda	<code>number</code>
<code>elettrica</code>	tem motor?	<code>boolean</code>
<code>situacao</code>	um de poucos valores fixos	união de literais

Essa tabela é o seu rascunho de tipo. Sem ela, o passo 2 vira chute.

Passo 2 · escreva o tipo

Cada linha da tabela vira uma linha da `interface`. Nada de lógica ainda:

```
interface Bicicleta {  
  id: string;  
  aro: number;  
  eletrica: boolean;  
  situacao: 'LIVRE' | 'EMPRESTADA';  
}
```

Duas decisões que aparecem toda hora:

- **valor que só pode ser um de poucos textos fixos:** união de literais (`'LIVRE' | 'EMPRESTADA'`), nunca `string` solta. O compilador passa a recusar valor inventado;
- **conjunto fixo que o código inteiro consulta:** `enum`, como os que já vêm prontos em `tipos.ts`. Se existe `Sala.IMAX`, escrever `'IMAX'` na mão é **magic string**: funciona hoje e apodrece amanhã, e o guia de tipagem avisa.

Regra de bolso: o tipo descreve **a forma** do dado. Se você está pensando em `if`, voltou a pensar em lógica cedo demais.

Passo 3 · assine a função

Com o tipo pronto, escreva a assinatura: **entrada tipada, saída tipada, corpo quase vazio**.

```
export function calcularAluguel(bicicleta: Bicicleta, horas: number): number {  
    return 0;  
}
```

Isso já compila, e esse é o ponto: o contrato existe antes da lógica. Agora rode:

```
npm run validar
```

O **guia de tipagem** confere parâmetro, retorno e as variáveis do corpo. A meta é o guia ficar **limpo agora**, antes de qualquer regra de negócio. Se apontar pendência, o problema está no passo 2 ou na assinatura, volte lá.

Dentro do corpo, a mesma disciplina: toda `const` e `let` nasce com tipo (`const total: number = 0;`).

Passo 4 · só agora a lógica

Estrutura de pé, dados correndo por ela. Agora sim:

```
npm test
```

- pegue **uma regra por vez**: o nome de cada teste diz qual regra ele cobra;
- teste vermelho é **mapa**, não bronca: leia o `expect`, veja o que entrou e o que deveria sair;
- terminou uma regra, rode de novo. A questão fica verde teste a teste, não de uma vez;
- o guia de tipagem tem que **continuar limpo**: lógica nova não é desculpa para `any`.

Quando a questão inteira ficar verde, suba um degrau: a próxima questão da escada recomeça no passo 1.

Sintomas de etapa pulada

o que aparece	qual etapa você pulou
<code>any</code> , ou variável sem tipo no corpo	passo 2: a forma do dado não foi pensada
<code>'IMAX'</code> escrito na mão com <code>Sala.IMAX</code> existindo	passo 2: o conjunto fixo já tinha dono
o guia aponta parâmetro ou retorno sem tipo	passo 3: a assinatura saiu no improviso
o teste passa mas o <code>typecheck</code> reclama	passo 3: a lógica correu na frente do contrato
meia hora de lógica sem rodar nada	passo 4: os testes existem para rodar cedo

O `npm run validar` mostra tudo isso de uma vez: nota dos testes visíveis, guia de tipagem e typecheck. É o retrato da sua entrega antes de virar zip.